

Acorn Runner 作业新手指南

这份指南是给刚开始学 Python 的同学看的。它不是作业答案，而是帮你读懂 `Acorn Assignment v1.4.pdf`：这个作业要你做什么、应该从哪里开始、以及怎样一步一步完成这个迷宫游戏。

1. 这个作业到底要干嘛？

你要用 Python 3 做一个文字版迷宫游戏，名字叫 **Acorn Runner**。

游戏设定很简单：

- 玩家是一颗橡果，用字母 `A` 显示。
- 地图是一个二维迷宫，用 `.txt` 文件提供。
- 玩家从起点 `X` 出发。
- 玩家要走到终点 `Y`。
- 地图中会有墙、水桶、火、传送门等特殊格子。
- 玩家可以输入 `w/a/s/d/e/q` 控制行动。

最终你要提交三类东西：

1. **游戏本体**：玩家可以手动玩这个迷宫。
2. **自动解题器 solver**：程序自动找出从起点到终点的路径。
3. **测试和报告**：写测试用例，并简单分析 BFS / DFS 解题算法。

2. 游戏规则概览

2.1 地图符号

配置文件里的每个字符都代表一个格子。

字符	含义
A	玩家当前位置，显示用，不一定直接出现在原始地图里
空格 ' '	空地，可以走
X	起点
Y	终点
*	墙，不能穿过
W	水桶，踩上去后获得 1 个水桶
F	火，没有水桶会失败；有水桶则消耗 1 个水桶并灭火
1 到 9	传送门，同样数字必须成对出现
0	不是合法传送门

例子：

```
*X*****
*      2      *
* *** ** *****
* *  W*   1  *
* *****
*  2 *   ** *F*
* ** ***  F  *
* 1*****F  *
*****Y*
```

你需要把这种文本地图读入 Python，变成程序可以操作的数据结构。

2.2 玩家指令

输入	作用
w	向上走
a	向左走
s	向下走
d	向右走
e	等待一回合
q	退出游戏

注意：

- 指令大小写不敏感，也就是说 `W` 和 `w` 都应该可以表示向上。
- 如果输入非法指令，要打印：

```
Please enter a valid move (w, a, s, d, e, q).
```

3. 作业提供/要求的文件结构

PDF 建议你按下面这些文件组织程序：

```
run.py           # 手动运行游戏的入口
solver.py        # 自动解题器入口

game.py          # 游戏核心逻辑
player.py        # 玩家类
cells.py         # 各种格子类
cell.py          # 如果脚手架里有，也可能用于放基础 Cell

game_parser.py   # 读取并解析地图

grid.py          # 把地图转换成字符串显示
```

PDF 里有一个文件关系图，大概意思是：

```

run.py      solver.py
 \         /
  \       /
   game.py
  /  |  |  \
grid player cells game_parser

```

也就是说：

- `run.py` 和 `solver.py` 是入口。
- 它们都需要使用 `game.py`。
- `game.py` 会用到地图、玩家、格子、解析器等模块。

4. 哪些东西不能乱改？

作业会自动测试一些函数和属性，所以这些名字必须保留。

重点包括：

文件	名字	为什么重要
<code>game_parser.py</code>	<code>parse(lines)</code>	自动测试会直接调用它
<code>grid.py</code>	<code>grid_to_string(grid, player)</code>	自动测试会直接调用它
<code>cells.py</code>	<code>display</code> 属性	测试会检查格子的显示字符
<code>player.py</code>	<code>display</code> 属性	显示玩家用
<code>player.py</code>	<code>num_water_buckets</code>	表示水桶数量
<code>player.py</code>	<code>row</code>	玩家所在行
<code>player.py</code>	<code>col</code>	玩家所在列

新手最容易犯的错是：

- 把函数名改了。
- 把参数顺序改了。
- 把 `display` 写成方法而不是属性。
- `parse()` 返回了字符串，而不是二维列表。
- `grid_to_string()` 直接 `print()`，但作业要求它返回字符串。

5. 最推荐的新手完成顺序

不要一上来就写完整游戏。建议按下面顺序做。

第 1 步：先完成 milestone 部分

Milestone 主要测这些：

1. `game_parser.py` 里的 `parse()`
2. `grid.py` 里的 `grid_to_string()`
3. `player.py` 里的基础属性：
 - `row`
 - `col`
 - `num_water_buckets`

这是最适合新手开始的地方。

第 2 步：先让地图能被读取

你需要先理解地图是一个二维表格。

比如配置文件：

```
***
X Y
***
```

可以理解成：

```
[
    ['*', '*', '*'],
    ['X', ' ', 'Y'],
    ['*', '*', '*']
]
```

但是作业更推荐你把每个字符变成一个 Cell 对象，例如：

```
[
    [WallCell(), WallCell(), WallCell()],
    [StartCell(), AirCell(), EndCell()],
    [WallCell(), WallCell(), WallCell()]
]
```

新手可以先从简单版本开始：

1. 先遍历每一行字符串。
2. 去掉末尾的 `\n`。
3. 遍历每个字符。
4. 根据字符创建对应的 Cell。
5. 返回二维列表。

第 3 步：写各种 Cell

你可以在 `cells.py` 里写不同的格子类。

最简单的思路是：每个格子类都有一个 `display` 属性。

例如：

```
class WallCell:
    def __init__(self):
        self.display = '★'
```

类似地，你会需要：

```
AirCell      display = ' '
StartCell    display = 'X'
EndCell      display = 'Y'
WallCell     display = '★'
WaterCell    display = 'W'
FireCell     display = 'F'
TeleportCell display = '1' 到 '9'
```

如果你刚学 Python，不一定要一开始就追求复杂继承。先写清楚、能跑、能测试更重要。

第 4 步：写 Player

`player.py` 里的 `Player` 至少应该记录：

```
self.display = 'A'
self.row = 起始行
self.col = 起始列
self.num_water_buckets = 0
```

玩家移动时，本质上就是改变 `row` 和 `col`。

例如：

指令	row 变化	col 变化
w	-1	0
s	+1	0
a	0	-1
d	0	+1
e	0	0

新手可以先写一个辅助函数，把输入转换成坐标变化。

第 5 步：写 grid_to_string()

`grid_to_string(grid, player)` 的任务是：

- 接收一个二维地图 `grid`
- 接收一个玩家对象 `player`
- 返回完整地图字符串
- 玩家所在位置显示为 `A`
- 最后显示水桶数量

注意，它应该返回字符串，不应该直接打印。

输出大概像这样：

```
*A*
* *
*Y*

You have 0 water buckets.
```

新手建议写法：

- 准备一个空字符串 `result`。
- 用双重循环遍历每一行、每一列。
- 如果当前位置等于玩家位置，加入 `player.display`。
- 否则加入当前 cell 的 `display`。
- 每一行结束后加 `\n`。
- 最后加空行和水桶信息。

特别注意单复数：

```
You have 0 water buckets.  
You have 1 water bucket.  
You have 2 water buckets.
```

6. 游戏核心逻辑怎么想？

游戏每一回合都可以拆成这几个动作：

```
显示地图  
读取玩家输入  
判断输入是否合法  
计算目标位置  
判断目标格子是什么  
根据格子规则更新游戏状态  
判断是否胜利/失败/继续
```

这就是 `game.py` 应该负责的主要内容。

7. 各种格子的处理规则

7.1 墙 Wall

玩家撞墙时：

- 玩家位置不变。
- 不记录这次非法移动。
- 打印消息：

```
You walked into a wall. Oof!
```

如果玩家走出地图边界，也当作撞墙处理。

7.2 水桶 Water

玩家踩到 `W`：

- 水桶数量加 1。
- 这个格子之后应当变成空地。
- 打印消息：

Thank the Honourable Furious Forest, you've found a bucket of water!

7.3 火 Fire

玩家踩到 **F** 有两种情况。

如果有水桶：

- 水桶数量减 1。
- 火被扑灭，格子变成空地。
- 玩家可以走过去。
- 打印消息：

With your strong acorn arms, you throw a water bucket at the fire. You acorn roll your way through the extinguished flames!

如果没有水桶：

- 游戏失败。
- 打印消息：

You step into the fires and watch your dreams disappear :(.

7.4 传送门 Teleport

传送门是数字 **1** 到 **9**。

规则：

- 相同数字必须正好出现两次。
- 玩家踩到其中一个，会被传送到另一个。
- 打印消息：

Whoosh! The magical gates break Physics as we know it and opens a wormhole through space and time.

特殊注意：

- 如果玩家已经在传送门上，输入 **e** 等待，也会再次传送。
- 如果玩家在传送门上撞墙，被推回原位时，不应该再次触发传送。

7.5 终点 End

玩家到达 Y，游戏胜利。

胜利时要打印指定结尾文本，包括：

- 胜利故事文本
- 总移动次数
- 移动列表
- YOU WIN!

8. 怎么记录移动？

你需要记录玩家有效走过的指令。

比如：

```
s, s, d, d
```

注意：

- 撞墙不应该记录。
- 走出边界不应该记录。
- 非法输入不应该记录。
- 正常移动、水桶、灭火、传送、到终点，应该记录。
- 失败那一步如果是走进火，也应该记录，因为玩家确实移动了。

作业输出里会用：

```
You made 2 moves.  
Your moves: s, s
```

如果只有一步，示例里可能是：

```
You made 1 move.  
Your move: s
```

建议你写一个小函数专门格式化移动记录，避免到处复制逻辑。

9. run.py 应该做什么？

`run.py` 是手动玩游戏的入口。

它大概应该负责：

1. 检查命令行参数。
2. 读取地图文件。
3. 创建 `Game` 对象。
4. 显示初始地图。
5. 循环读取用户输入。
6. 调用游戏逻辑处理移动。
7. 打印消息。
8. 如果胜利、失败或退出，就结束程序。

命令格式大概是：

```
python3 run.py <filename>
```

如果没有参数，打印：

```
Usage: python3 run.py <filename> [play]
```

PDF 提到可以用可选参数 `play` 开启清屏效果，但这不是重点，新手可以先跳过。

10. `game_parser.py` 应该做什么？

这是 milestone 重点。

`parse(lines)` 接收的是字符串列表，比如：

```
[
    '***\n',
    'X Y\n',
    '***\n'
]
```

它要返回二维 cell 列表。

同时要检查错误。

10.1 非法字符

如果出现未知字符，比如 `z`，要抛出：

```
ValueError('Bad letter in configuration file: Z.')
```

注意：0 也是非法传送门，所以也应该报错。

10.2 起点数量错误

如果 x 不是正好 1 个，要抛出：

```
ValueError('Expected 1 starting position, got <number>.'
```

10.3 终点数量错误

如果 y 不是正好 1 个，要抛出：

```
ValueError('Expected 1 ending position, got <number>.'
```

10.4 传送门不是成对出现

如果某个传送门数字不是正好两个，要抛出：

```
ValueError('Teleport pad <number> does not have an exclusively matching pad.'
```

新手建议：

- 用一个字典记录每个数字出现几次。
- 解析结束后再检查每个数字是不是正好 2 次。

例如：

```
teleport_counts = {  
    '1': 2,  
    '2': 1  
}
```

这里 2 就有问题。

11. solver.py 是什么？

solver.py 是自动解迷宫程序。

它接收：

```
python3 solver.py <filename> <mode>
```

其中 `<mode>` 是：

```
DFS  
BFS
```

如果没有参数，打印：

```
Usage: python3 solver.py <filename> <mode>
```

它需要输出：

```
Path has <number> moves.  
Path: <moves comma delimited>
```

如果没有路：

```
There is no possible path.
```

12. 新手怎么理解 BFS 和 DFS?

12.1 BFS：广度优先搜索

BFS 像一圈一圈扩散。

如果起点在中间，它会先尝试：

```
走 1 步能到哪里？  
走 2 步能到哪里？  
走 3 步能到哪里？  
...
```

所以 BFS 的优点是：

- 如果每一步成本一样，BFS 可以找到最短路径。
- 作业要求 BFS 输出 optimal path，也就是最短路径。

缺点是：

- 可能需要保存很多状态。
- 地图大时会比较占内存。

12.2 DFS：深度优先搜索

DFS 像一直沿一条路走到底，走不通再回头。

优点是：

- 思路简单。
- 占用内存可能比 BFS 少。
- 有时终点刚好在它先尝试的方向上，会很快。

缺点是：

- 不保证最短路径。
- 可能绕很远。
- 如果不小心处理，会在循环里卡住。

作业说：

- BFS 必须找最短路径。
- DFS 只需要找一条有效路径。

13. solver 最关键的难点：状态不只是位置

普通迷宫只需要记录：

```
(row, col)
```

但这个游戏不够。

因为同一个位置，可能因为状态不同而结果不同。

例如：

你现在在同一个格子上：

- 情况 A：你有 0 个水桶
- 情况 B：你有 1 个水桶

这两个状态完全不同。因为有水桶时你可以过火，没有水桶时不能过火。

所以 solver 里的 visited 不能只记录位置。更合理的是记录：

```
(row, col, num_water_buckets, 地图变化情况)
```

地图变化情况包括：

- 哪些水桶已经被拿走了。
- 哪些火已经被扑灭了。

如果你只记录 (row, col)，可能会错误地认为“这个格子来过了，不用再来”，从而错过正确路径。

新手可以先实现游戏本体，再做 solver。solver 是最后的大块。

14. 建议的新手实现路线图

下面是一条比较稳的路线。

阶段 A：最小可运行版本

目标：不考虑水、火、传送，只让玩家能在地图里走。

完成：

1. Cell 类和几个基础格子。
2. parse(lines) 能读地图。
3. Player 能记录位置。
4. grid_to_string() 能显示地图。
5. Game 能移动玩家。
6. 撞墙不能动。
7. 走到 Y 胜利。

测试地图：

```
***  
X Y  
***
```

你应该能输入 d、d 到达终点。

阶段 B：加入水桶和火

目标：实现 `W` 和 `F`。

完成：

1. 踩到 `W` 后水桶 +1。
2. `W` 被拿走后变空地。
3. 有水桶时可以灭火，水桶 -1。
4. 没水桶踩火则失败。

测试地图：

```
*****
XWIFY
*****
```

玩家输入：

```
d, d, d
```

应该先拿水，再灭火，再到终点。

阶段 C：加入传送门

目标：实现数字传送。

完成：

1. `parse()` 检查每个传送门数字必须正好两个。
2. 玩家踩到一个 `1`，传送到另一个 `1`。
3. `e` 等待时，如果站在传送门上，会再次传送。
4. 撞墙返回传送门时，不重复触发。

测试地图：

```
*****
X1 1Y
*****
```

阶段 D：完善 `run.py` 输出

目标：输出格式尽量贴合作业示例。

完成：

1. 参数错误提示。
2. 非法输入提示。
3. 胜利输出。
4. 失败输出。
5. 退出输出 Bye! 。
6. 移动次数和移动列表。

阶段 E：写测试

作业有 10% 是学生自己的测试。

你需要写：

1. 单元测试：测每个模块。
2. 端到端测试：模拟完整游戏输入输出。

建议测试这些情况：

- 简单胜利。
- 走进墙。
- 走出边界。
- 水桶数量单复数。
- 拿水桶。
- 有水桶灭火。
- 无水桶踩火失败。
- 非法输入。
- 退出游戏。
- 传送门。
- 传送门数量不匹配。
- 没有起点。
- 多个起点。
- 没有终点。
- 非法字符。

阶段 F：做 solver

目标：自动找路径。

先做 BFS，再做 DFS。

建议先把“尝试一步”的逻辑复用游戏本体，或者写一个不会打印的模拟函数。

solver 的核心数据可以这样理解：

一个搜索状态 = 当前游戏局面 + 走到这里的路径

BFS 使用队列：

先进先出
先加入的状态先探索

DFS 使用栈：

后进先出
最后加入的状态先探索

PDF 提示说：BFS 和 DFS 最好只需要改一行。这通常意味着你可以写同一个搜索函数，只是取下一个状态的方法不同。

例如概念上：

```
if mode == 'BFS':  
    current = frontier.pop(0)  
else:  
    current = frontier.pop()
```

但注意：真实代码要考虑效率和状态复制。

15. 新手最容易踩的坑

坑 1：函数直接 print，没有 return

`grid_to_string()` 必须返回字符串。

错误思路：

```
print(result)
```

正确思路：

```
return result
```

`run.py` 再负责打印。

坑 2：忘记处理换行符

读取文件时，每行末尾可能有 `\n`。

如果不去掉，`parse()` 可能会把 `\n` 当成非法字符。

可以使用：

```
line = line.rstrip('\n')
```

不要用会删除地图空格的方式，例如盲目使用 `strip()`，因为地图中的空格本身代表 Air Cell。

坑 3：把空格删掉了

地图里的空格是有效格子。

例如：

```
X Y
```

中间那个空格不是“没用的空白”，而是玩家可以走的路。

所以不要对整行使用会删除左右空格的处理，除非你非常确定不会影响地图。

坑 4：没有区分“合法移动”和“非法移动”

撞墙不是有效移动。

所以：

- 玩家位置不变。
- 不加入 moves。
- 不增加 move count。

坑 5：solver 只记录位置

因为水桶、火、传送门会改变游戏状态，所以 visited 只写 `(row, col)` 可能是错的。

更安全的是把玩家状态和地图变化也纳入 visited。

坑 6：输出文字不完全一致

自动测试很可能比较输出文本。

所以这些消息最好从 PDF 复制，不要自己改写：

- 撞墙消息
- 找到水桶消息
- 灭火消息
- 失败消息
- 传送消息
- 胜利/失败结尾
- usage 消息

16. 报告要写什么？

报告最多 500 字，PDF 要求分两部分。

Section 1: Testing

要回答：

1. 为什么写测试是好事？列 3 个原因。
2. 什么是 mock？mock 的优缺点是什么？一般用于单元测试还是 E2E 测试？
3. 找一个现实案例：测试不足导致问题，并引用资料。

Section 2: Solver

要回答：

1. BFS 的优缺点，什么时候用 BFS？
2. DFS 的优缺点，什么时候用 DFS？
3. 如果终点离起点很近，DFS 一定比 BFS 快吗？
4. 为什么这个游戏的 visited 不能只是 visited cells？什么变化会让同一格子变成不同状态？

17. 一个适合新手的每日计划

如果你时间比较紧，可以这样安排。

第一天

- 读懂地图符号。

- 写 `cells.py`。
- 写 `player.py`。
- 写 `parse()` 的基础版本。

第二天

- 写 `grid_to_string()`。
- 让 milestone 测试尽量通过。
- 写最简单移动逻辑。

第三天

- 加入墙、水桶、火。
- 写胜利和失败输出。
- 完成 `run.py`。

第四天

- 加入传送门。
- 修复各种边界情况。
- 写自己的测试。

第五天

- 写 BFS solver。
- 写 DFS solver。
- 检查输出格式。
- 写报告。

18. 最小开发建议

对于刚学 Python 的同学，建议遵守这几个原则：

1. 先让小地图跑通，再跑大地图。
2. 先实现普通移动，再实现特殊格子。
3. 先完成 milestone，再挑战 solver。
4. 每完成一个功能，马上写一个小测试。
5. 不要一次写太多代码。写一点，运行一点，确认没错再继续。
6. 输出文字要和 PDF 一致。自动测试通常很严格。
7. 不要公开发布你的完整代码。PDF 明确提醒不要在公开平台贴代码。

19. 你可以用这张总流程图检查自己

```
读取地图文件
↓
parse(lines) 解析成 grid
↓
找到起点 X, 创建 Player
↓
显示 grid_to_string(grid, player)
↓
读取输入 w/a/s/d/e/q
↓
计算目标位置
↓
判断目标格子
  |— 墙: 不动, 提示撞墙
  |— 空地: 移动
  |— 水桶: 移动, 水桶 +1, 格子变空地
  |— 火: 有水则灭火; 无水则失败
  |— 传送门: 移动并传送到配对位置
  |— 终点: 胜利
↓
继续下一回合, 或结束游戏
```

20. 最后总结

这份作业看起来很长，但本质上是几个基础能力的组合：

- 读取文件
- 处理字符串
- 使用二维列表
- 写类和对象
- 根据条件判断游戏规则
- 循环读取用户输入
- 写测试
- 用 BFS / DFS 搜索路径

如果你是 Python 新手，最重要的是不要一开始就想“一次写完整个游戏”。你应该把它拆成很多小目标：

能读地图 → 能显示地图 → 能移动 → 能处理墙 → 能处理水和火 → 能处理传送 → 能胜利失败 → 能自动解题

每一步都能单独测试。这样这个作业就会从“一个很大的游戏项目”，变成一组可以逐个完成的小练习。